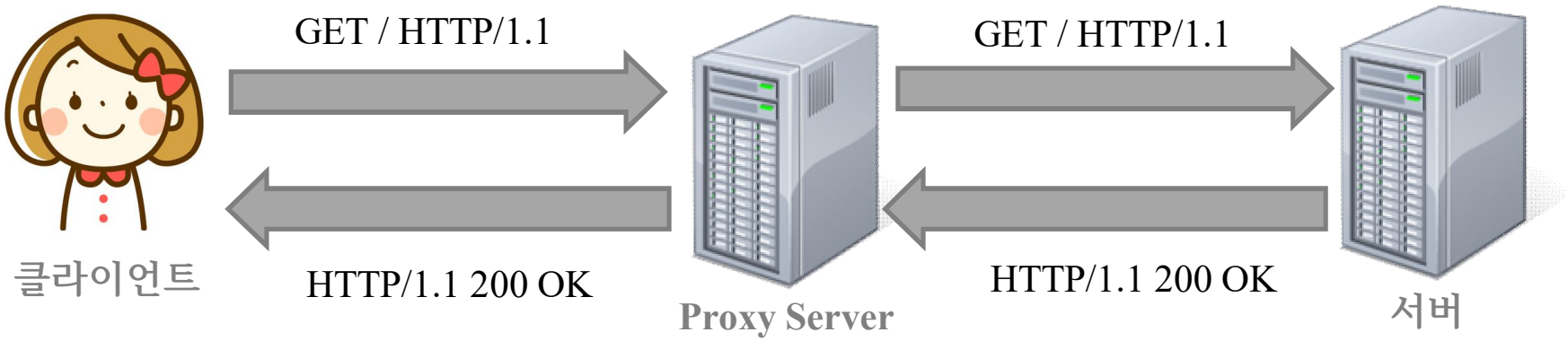
Web Proxy

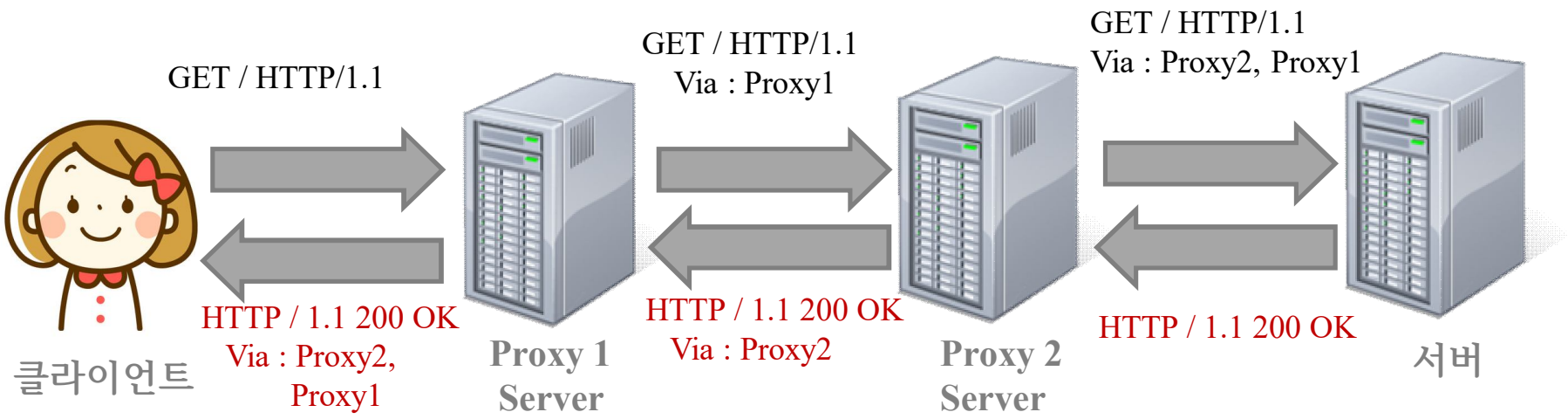
■ 서버와 클라이언트의 양쪽 역할을 중계 프로그램

- 클라이언트의 Request를 서버에 전송

- 서버로부터의 Response를 클라이언트에 전송



Web Proxy


GET / HTTP/1.1
GET / HTTP/1.1
Via : Proxy1
GET / HTTP/1.1
Via : Proxy2, Proxy1
클라이언트
HTTP / 1.1 200 OK
Via : Proxy2,
Proxy1
Proxy 1
Server
HTTP / 1.1 200 OK
Via : Proxy2
Proxy 2
Server
HTTP / 1.1 200 OK
서버

HTTP

- Request Method 종류

  - 서버에 임무를 부여하는 HTTP 메소드

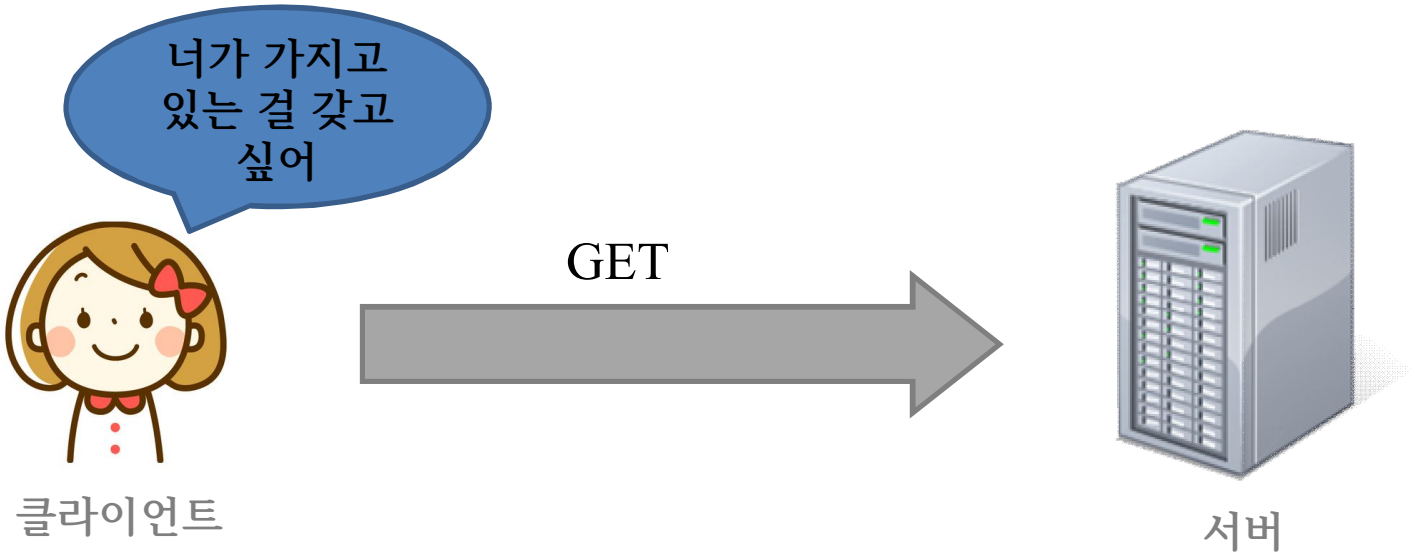
- GET
- POST
- HEAD
- OPTIONS
- PUT
- DELETE
- TRACE

```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

HTTP

HTTP Request : Get 방식

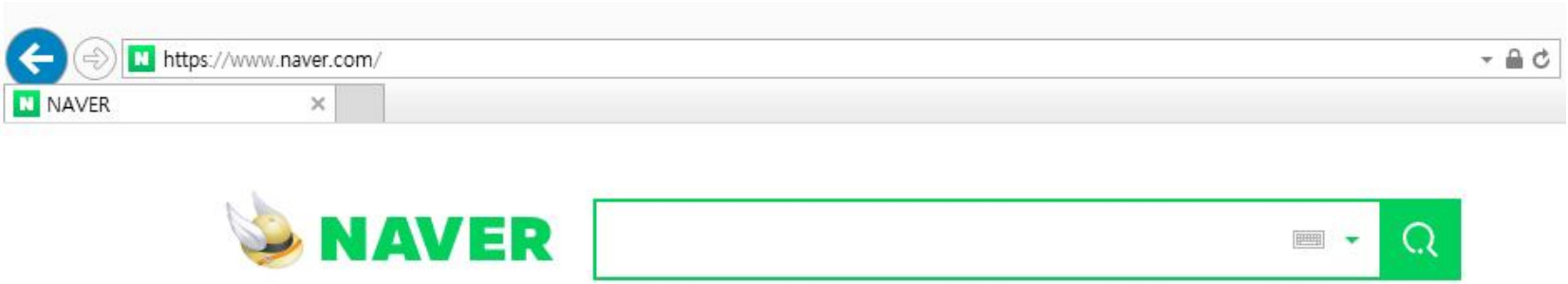
- 가장 일반적인 HTTP Request 형태
- 요청 데이터의 인수를 웹 브라우저의 URL로 전송함
- Request URI로 식별된 리소스를 가져올 수 있도록 요구



HTTP

HTTP Request : Get 방식

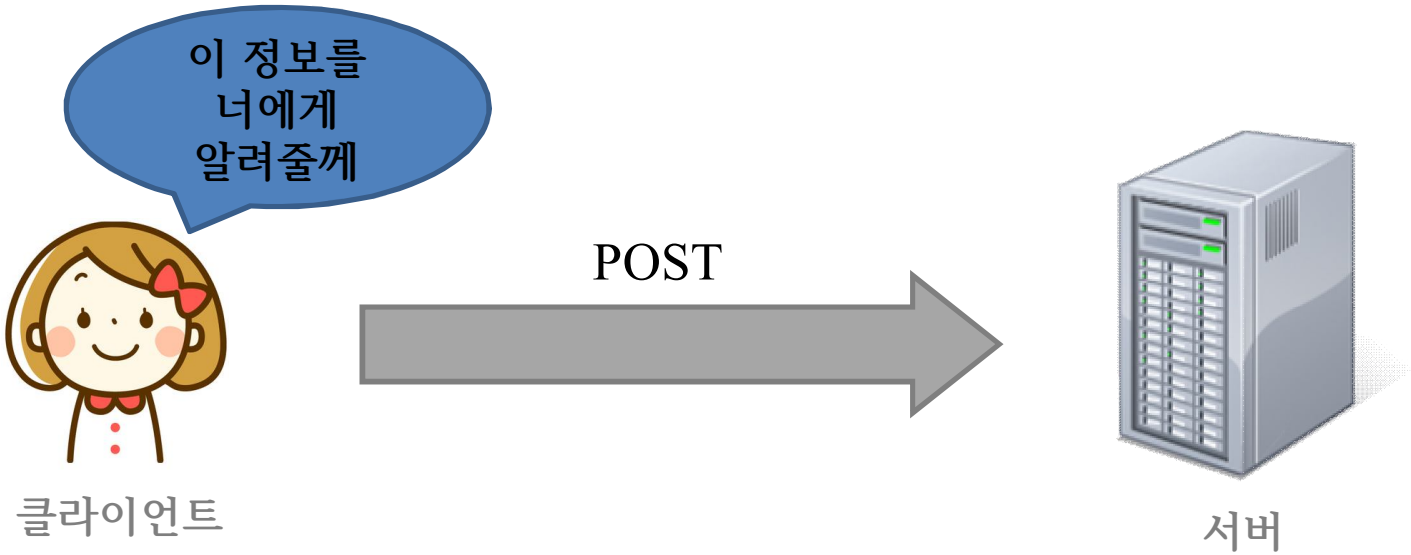
- 데이터가 주소 입력란에 표시되므로 최소한의 보안도 유지되지 않는 취약한 방식



HTTP

HTTP Request : POST 방식

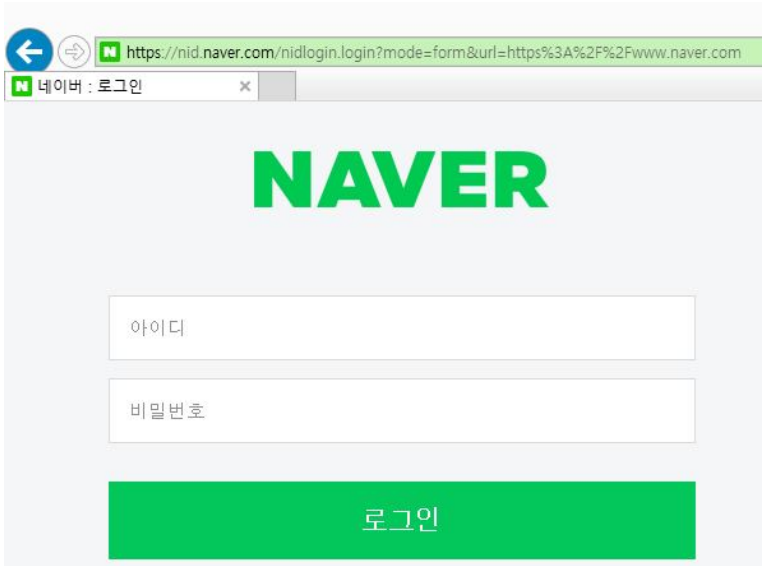
- 엔티티를 전송하기 위해 사용
- GET으로도 엔티티를 전송할 수 있지만, 자주 사용하지 않고 일반적으로 POST를 사용



HTTP

HTTP Request : POST 방식

- URL에 요청 데이터를 기록하지 않고 HTTP 헤더에 데이터를 전송.
- 엔티티를 URL로 전송하지 않으므로 다른 사용자가 링크로 해당 페이지를 볼 수 없음

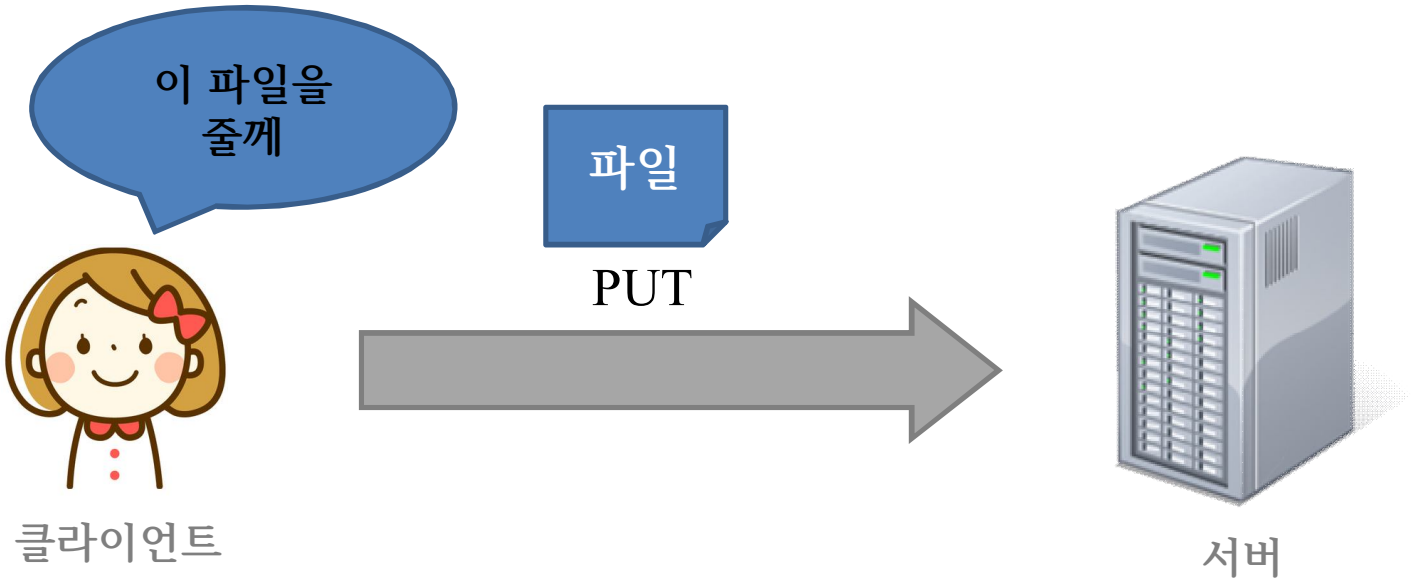


게시판의 경우, 목록이나 글 보기 화면은 접근 자유도를 위해 GET 방식을 사용, 게시글을 저장·수정·삭제하거나 대용량 데이터를 전송할 때는 POST 방식을 사용.

HTTP

HTTP Request : PUT 방식

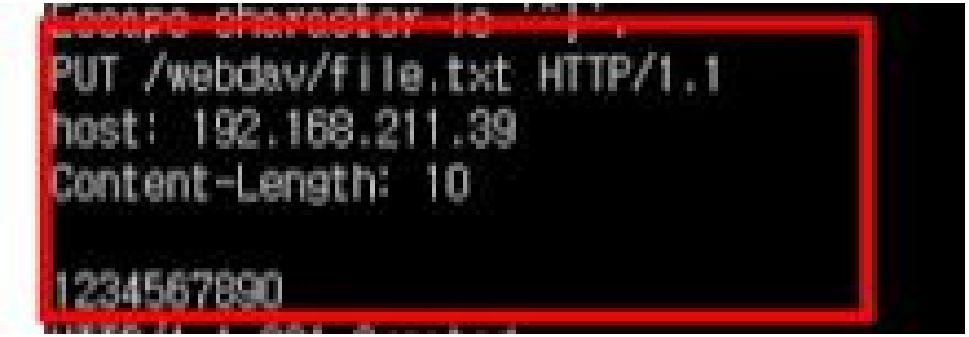
- POST와 유사한 전송 구조를 가짐
- 파일을 전송하기 위해 사용
- Request 중에 포함된 엔티티를 Request URI로 지정한 곳에 보존하도록 요구



HTTP

HTTP Request : PUT 방식

- 홈페이지 변조에 많이 악용되고 있음

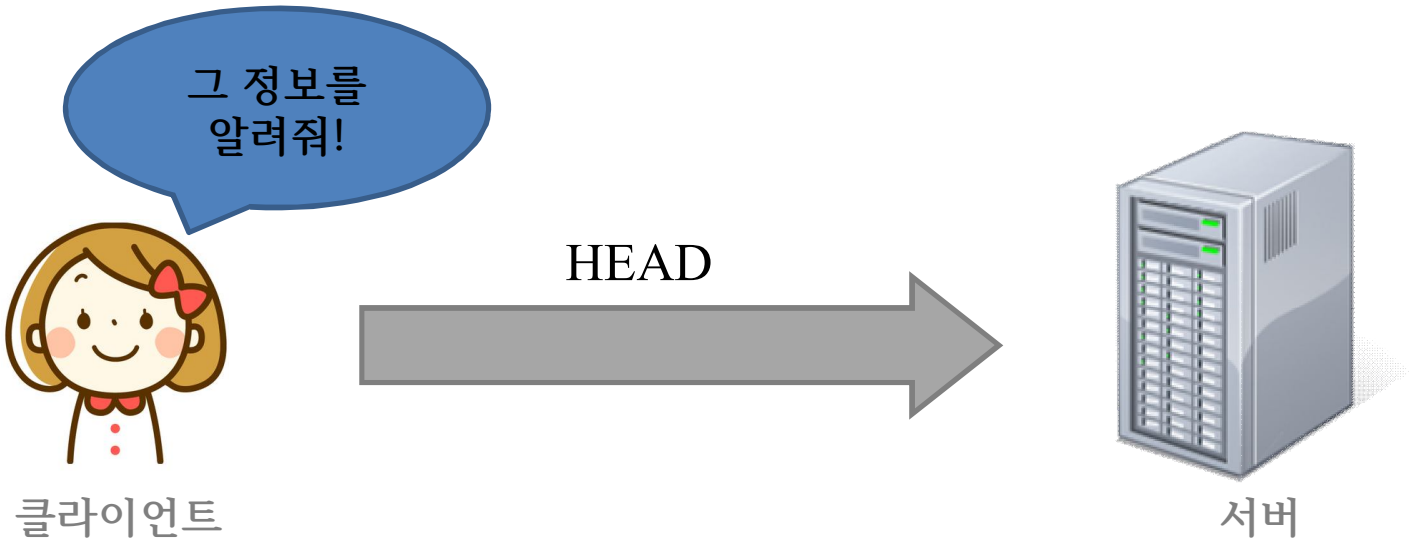


- PUT 방식으로 전달되는 URI는 파일의 이름 또는 경로
- 클라이언트가 지정한 위치에 특정 파일을 가져다 놓으려고 할 때 PUT 방식 사용
- 1234567890 이라는 내용이 입력된 file.txt 파일을 웹서버로 업로드

HTTP

HTTP Request : Head 방식

- GET방식과 동일
- 서버 측 데이터를 검색하고 요청하는 데 사용
- BODY가 없고 응답코드와 HEAD로만 응답
- URI 유효성과 리소스 갱신 시간을 확인하는 목적을 사용
- 웹 서버 정보확인, 헬스체크, 버젼확인, 최종 수정일자 확인 용도

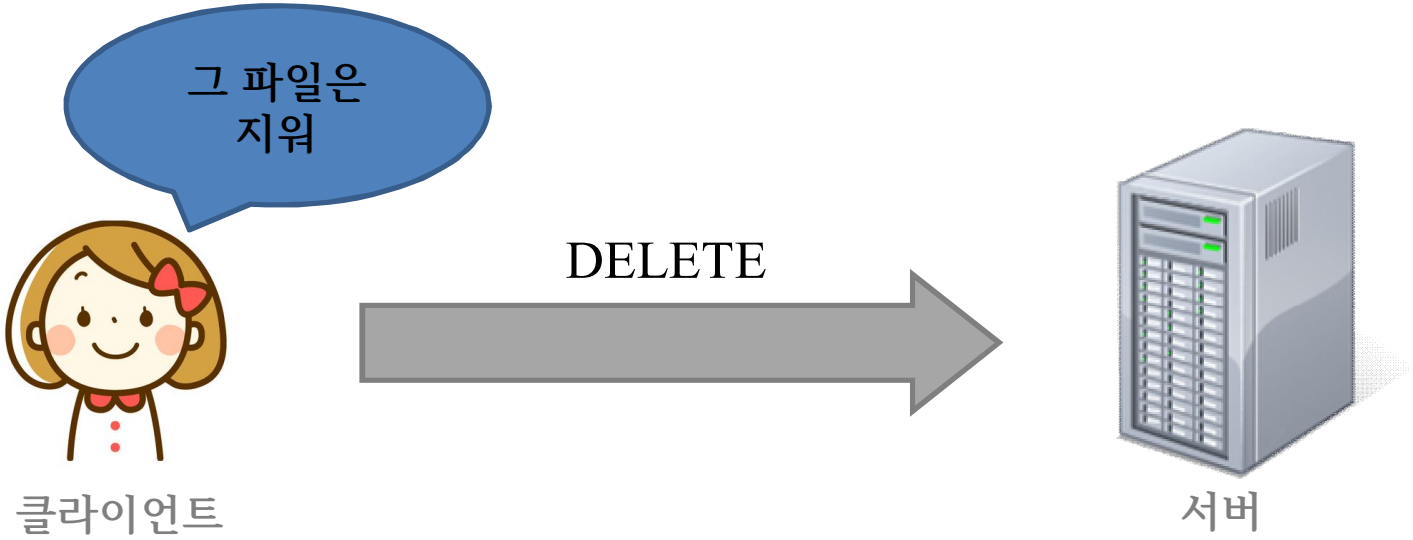


HTTP

HTTP Request : Delete 방식

- URI에 지정된 자원을 서버에서 지울 수 있게 함
- 원격지 웹 서버에 파일을 삭제하기 위해 사용되며 PUT과는 반대 메소드

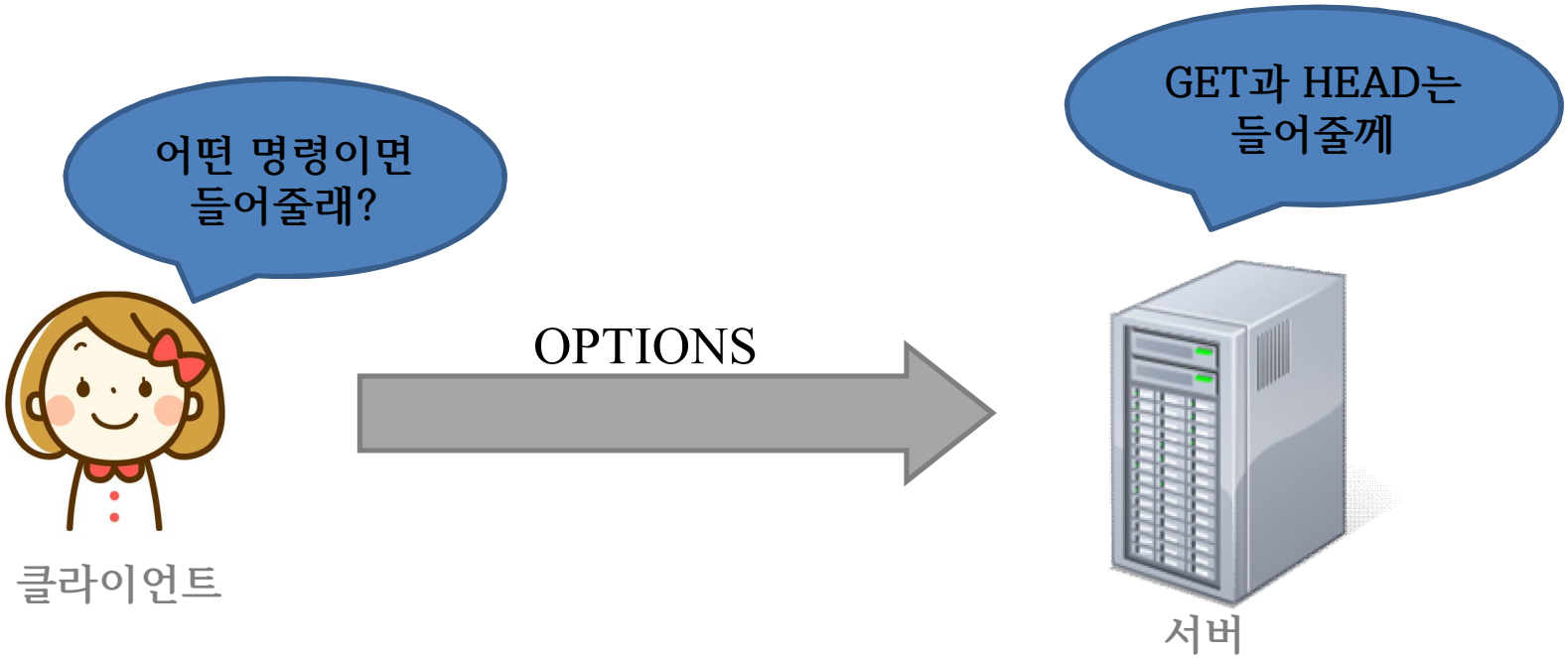
```
DELETE /webdav/file.txt HTTP/1.1
host: 192.168.211.39
```



HTTP

HTTP Request : Option 방식

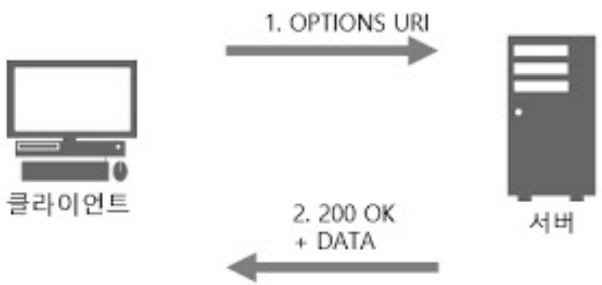
- 자원에 대한 요구/응답 관계와 관련된 선택 사항 정보를 요청할 때 사용
- 시스템에서 지원되는 메소드 종류를 확인 할 수 있음
- 웹서버에서 지원하는 HTTP 요청방식을 확인하고자 할때 사용



HTTP

HTTP Request : Option 방식

- 자원에 대한 요구/응답 관계와 관련된 선택 사항 정보를 요청할 때 사용
- 시스템에서 지원되는 메소드 종류를 확인 할 수 있음
- 웹서버에서 지원하는 HTTP 요청방식을 확인하고자 할때 사용



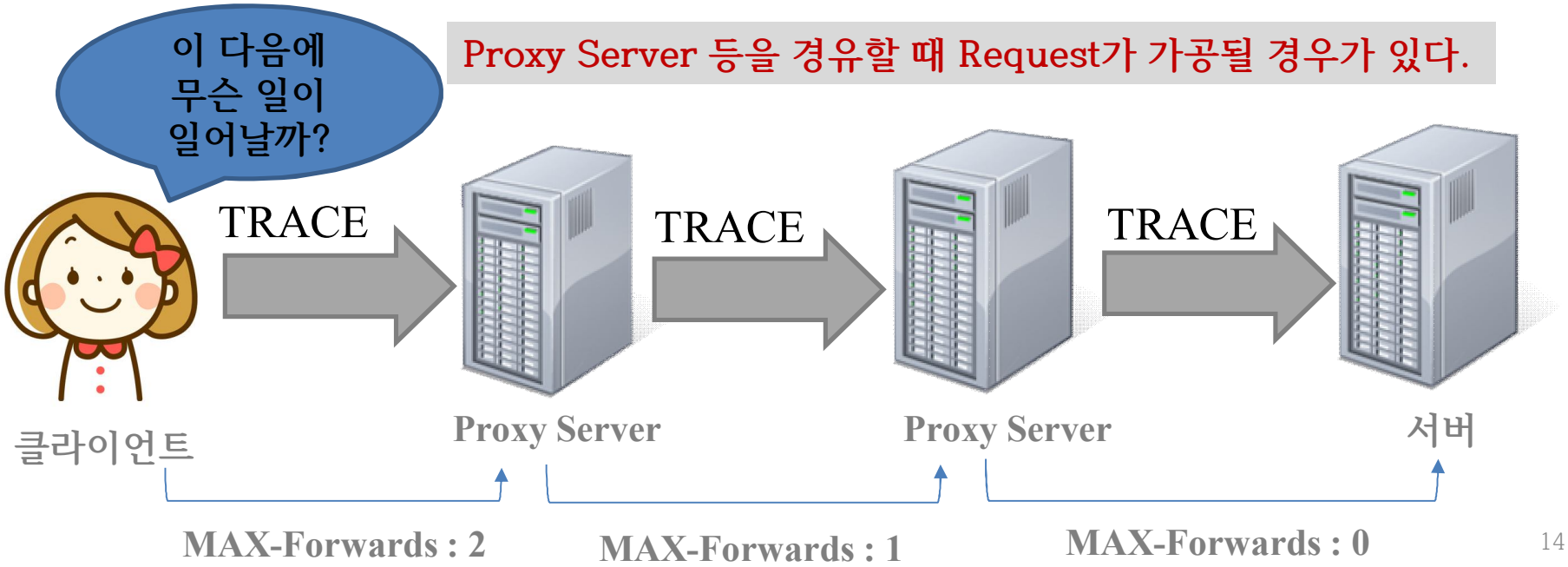

```
[root@AV2 ~]# telnet 192.168.211.39 80
Trying 192.168.211.39...
Connected to 192.168.211.39.
Escape character is '^]'.
OPTIONS /index.html HTTP/1.1
Host: 192.168.211.39

HTTP/1.1 200 OK
Date: Tue, 04 Jul 2017 00:28:01 GMT
Server: Apache/2.4.25 (Ubuntu)
Allow: GET,HEAD,POST,OPTIONS,HEAD,HEAD,TRACE
Content-Length: 0
Content-Type: text/html
```

HTTP

HTTP Request : Trace 방식

- 요구 메시지의 최종 수신처까지 루프백을 검사하는 데 사용
- 웹 서버로 가는 네트워크 경로를 체크하는 메소드
- 웹서버와 클라이언트 사이에 중간서버(웹프록시 또는 웹캐시서버등)가 존재할 경우 TRACE 방식을 이용하여 확인할 수 있음



HTTP

HTTP Request : Trace 방식

클라이언트

중간 서버

Wireshark · Follow HTTP Stream (tcp.stream eq 0) · trace

```
TRACE http://192.168.211.39/index.html HTTP/1.1
Host: 192.168.211.39

HTTP/1.1 200 OK
Date: Mon, 03 Jul 2017 08:57:35 GMT
Server: Apache/2.4.25 (Ubuntu)
Content-Type: message/http
X-Cache: MISS from Squid
X-Cache-Lookup: NONE from Squid:3128
Transfer-Encoding: chunked
Via: 1.1 Squid (squid/3.5.23)
Connection: keep-alive

TRACE /index.html HTTP/1.1
Host: 192.168.211.39
Via: 1.1 Squid (squid/3.5.23)
X-Forwarded-For: 192.168.211.160
Cache-Control: max-age=259200
Connection: keep-alive
```

서버

Wireshark · Follow HTTP Stream (tcp.stream eq 0) · trace-srv

```
TRACE /index.html HTTP/1.1
Host: 192.168.211.39
Via: 1.1 Squid (squid/3.5.23)
X-Forwarded-For: 192.168.211.160
Cache-Control: max-age=259200
Connection: keep-alive

HTTP/1.1 200 OK
Date: Mon, 03 Jul 2017 08:57:35 GMT
Server: Apache/2.4.25 (Ubuntu)
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: message/http

TRACE /index.html HTTP/1.1
Host: 192.168.211.39
Via: 1.1 Squid (squid/3.5.23)
X-Forwarded-For: 192.168.211.160
Cache-Control: max-age=259200
Connection: keep-alive
```

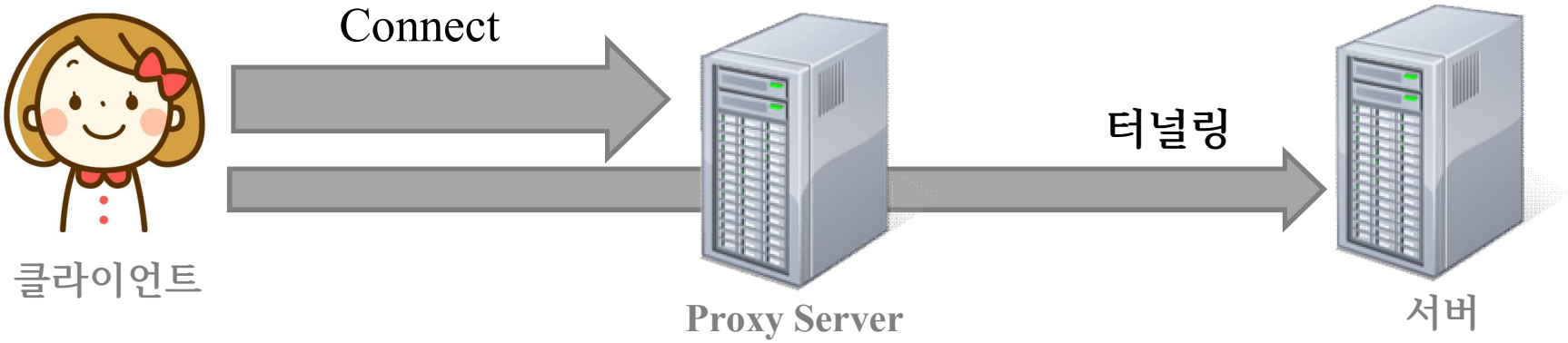
중간서버에서 추가된 Via 헤더

- 거의 사용되지 않는 메소드

HTTP

HTTP Request : Connect 방식

- 프록시에 터널 접속 확립을 요함으로써 TCP 통신을 터널링 하기 위해 사용
- SSL/TLS 프로토콜로 암호화 된 것을 터널링 시키기 위해 사용




```
CONNECT proxy.test.com:8080 HTTP /1.1
Host : proxy.testc.om
```